

Pinia 기초 Vue 3 상태 관리

Vue 3 공식 상태 관리 라이브러리 완전 정복



이번 강의 목표

- ✓ Pinia가 필요한 이유 이해
- ✓ Store / State / Getter / Action 개념
- ✓ 프로젝트 세팅 & 컴포넌트 연동
- ✓ 실전 코드 예제 확인

INDEX

강의 목차

Pinia의 기초부터
실전 활용까지
단계별 학습 로드맵

🕒 총 소요 시간: 약 15분

01

Intro – 왜 Pinia가 필요한가?

Props/Emit의 한계와 상태 관리의 필요성

02

Pinia 기본 개념

Store, State, Getter, Action의 역할

03

프로젝트 세팅 & Store 만들기

설치부터 defineStore 정의까지

04

컴포넌트에서 Store 사용하기

storeToRefs 활용 및 읽기/쓰기 실습

05

정리 & 확장 아이디어

핵심 요약 및 Todo 리스트 도전 과제



Why Pinia?

앱이 커질수록
복잡해지는 데이터 흐름,
어떻게 해결할까요?

기존 방식의 한계 (Props/Emit)



Prop Drilling 발생

상위 컴포넌트에서 하위로 데이터를 계속 전달해야 하는 번거로움



복잡한 형제 컴포넌트 공유

부모를 거쳐야만 형제끼리 데이터를 주고받을 수 있어 구조가 복잡해짐



Vue 3 공식 솔루션: Pinia



가볍고 간편함

복잡한 설정 없이 바로 사용 가능하며
가벼운 용량



Composition API

Vue 3의 setup 문법과 완벽하게 조화를
이루는 구조



Core Concepts

Pinia를 구성하는
3가지 핵심 요소와
데이터 흐름 이해하기



DEFINITION

Store = 중앙 상태 참고

여러 컴포넌트가 데이터를 공유하는 단일 진실 공급원(Single Source of Truth)

Store 구성 요소 (3 Pillars)



State

≡ `data()`

실제 관리되는 데이터
(예: count, user)



Getters

≡ `computed`

State를 기반으로 계산된 값
(예: doubleCount)

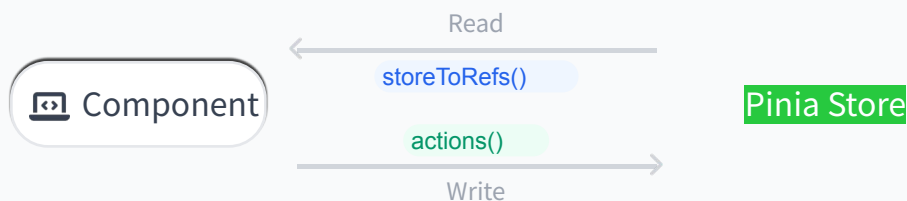


Actions

≡ `methods`

State를 변경하는 함수 (비동기 가능)
(예: increment)

USAGE FLOW



TODAY'S MISSION

- ✓ Counter Store 생성
- ✓ 2배수 Getter 구현
- ✓ 증감 Action 버튼 연동



PROJECT SETUP

프로젝트 세팅 & Store 만들기

1 패키지 설치

→ `npm install pinia`

2 플러그인 등록

`main.js` 에서 `createPinia()`를 호출하여 Pinia 인스턴스를 생성하고 앱에 등록합니다.

3 Store 정의

-  **state:** 실제 데이터
-  **getters:** 계산된 값
-  **actions:** 변경 함수

src/main.js

```
1 import { createApp } from 'vue'
2 import { createPinia } from 'pinia'
3 import App from './App.vue'
4
5 const app = createApp(App)
6 app.use(createPinia()) // Pinia 등록
7 app.mount('#app')
```

src/stores/counter.js

```
1 import { defineStore } from 'pinia'
2
3 export const useCounterStore = defineStore('counter', {
4   // 1. State
5   state: () => ({ count: 0 }),
6
7   // 2. Getters
8   getters: {
9     doubleCount: (state) => state.count * 2
10  },
11
12   // 3. Actions
13   actions: {
14     increment() { this.count++ }
15   }
16 })
```

컴포넌트에서 Store 사용하기

1

Store 인스턴스 가져오기

정의한 `useCounterStore` 혹은 호출하여 store 인스턴스를 생성합니다.

2

상태 읽기 (State/Getters)

⚠ 주의사항

일반 구조 분해 할당을 하면 **반응성이 깨집니다!** 반드시 `storeToRefs` 를 사용해야 합니다.

3

액션 호출 (Actions)

함수(Actions)는 상태가 아니므로 구조 분해 할당을 해도 안전하게 바로 사용할 수 있습니다.

```
src/App.vue

1 <script setup>
2 import { storeToRefs } from 'pinia'
3 import { useCounterStore } from '@/stores/counter'
4
5 // 1. Store 인스턴스 가져오기
6 const counter = useCounterStore()
7
8 // 2. State/Getters (반응성 유지 위해 storeToRefs 사용)
9 const { count, doubleCount } = storeToRefs(counter)
10
11 // 3. Actions (그냥 구조 분해 가능)
12 const { increment } = counter
13 </script>
14
15 <template>
16   <div>
17     <p>Count: {{ count }}</p>
18     <p>Double: {{ doubleCount }}</p>
19     <button @click="increment">+1</button>
20   </div>
21 </template>
```

정리 & 확장 아이디어

| 핵심 개념 다시보기



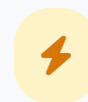
State

컴포넌트들이 공유하는 실제 데이터.
storeToRefs()로 반응성 유지



Getters

State를 기반으로 계산된 값. Vue의 computed와 유사한 역할



Actions

상태를 변경하는 함수. 동기/비동기 로직 모두 처리 가능

≡ 현업에서의 Store 분리

기능별로 Store를 나누어 관리하는 것이 일반적입니다.



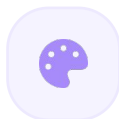
auth.js
로그인/유저

+



cart.js
장바구니

+



theme.js
다크모드

🚩 Challenge

Homework

</> **기능 확장하기**
state에 step을 추가하고, step만큼 증가하는 action 만들기

🔗 **컴포넌트 공유 확인**
새 컴포넌트(AnotherCounter.vue)를 만들고 같은 Store 연결해보기

☰ **Todo 리스트 만들기**
Pinia Store를 이용해 할 일 추가/삭제가 가능한 앱 구현해보기